

UltraEdit: Mapping Chords to JavaScript Scripts — Troubleshooting Guide

Scope: Mapping a multi-key chord (e.g. Ctrl+K, Ctrl+R) to a JavaScript script named `run_selection` in UltraEdit / UESTudio. Covers distinguishing the two dialogs where shortcuts are assigned, verifying the script registration, entering the chord, and debugging when the chord does not fire.

1. Know Which Dialog to Use

UltraEdit has two completely separate places where keyboard shortcuts get assigned. Using the wrong one is the most common cause of a chord that never triggers.

Scripting → Scripts dialog (the one you want for JavaScript)

Aspect	Detail
What it is for	Adding .js script files to UltraEdit and assigning a Hotkey and/or Chord to each script for quick execution (UltraEdit Wiki: Scripts).
How to open	Click All scripts in the Script group of the Advanced tab (ribbon UI), or use the Scripting → Scripts menu path in classic-menu builds (UltraEdit Wiki: Scripts).
What you assign here	The script's Hotkey field and Chord field — both typed directly into the script list row (UltraEdit Wiki: Scripts).
Key rule	A hotkey must be created first before a chord can be set up for a script (UltraEdit Scripting Engine Tutorial).

Advanced → Tool Configuration / Configure Tools (NOT for JavaScript scripts)

Aspect	Detail
What it is for	Defining user tools — external DOS or Windows commands/programs that interact with other applications. Up to 25 standard tools and 25 project tools can be configured (UltraEdit Wiki: Configure tools).
What it is NOT for	JavaScript scripts. A user tool runs a command line (with placeholders like %f for the active file path), not an internal UltraEdit JavaScript (UltraEdit Wiki: Configure tools).
Default shortcut	Tools get an automatic Ctrl+Shift+# shortcut where # is the tool number; custom shortcuts are assigned via Settings → Key Mapping , not in the Tool Configuration dialog itself (UltraEdit Wiki: Configure tools).
Common mistake	Adding the script path as a "tool" here will launch an external process (if the OS knows how to run the file), not execute it through UltraEdit's JavaScript engine. The chord will behave unpredictably or not at all.

Bottom line: If your deliverable is `run_selection.js` running inside UltraEdit's scripting engine, the chord must be assigned in the **Scripting → Scripts** dialog, not in **Tool Configuration**.

2. Preflight Checklist: Verifying `run_selection` Is Correctly Added

Before testing any chord, confirm the script itself is registered and runnable.

#	Check	How to verify	Pass condition
1	Script file exists on disk	Open the <code>.js</code> file directly in UltraEdit (File → Open).	File opens and displays JavaScript source.
2	Script is added to the Scripts list	Open Scripting → Scripts (or Advanced tab → All scripts). Look for <code>run_selection</code> in the script list.	Script name appears in the list.
3	Correct file path	Hover the cursor over the script name in the list. A tooltip shows the full path (UltraEdit Wiki: Scripts).	Path matches the intended <code>.js</code> file location.
4	Description is set	Double-click the Description field at the bottom of the dialog to edit it (UltraEdit Wiki: Scripts).	Description reads <code>run_selection</code> or similar identifying text.
5	Script runs manually	Click the script name at the bottom of the Scripting menu, or select it in the Scripts dialog and run it (UltraEdit Scripting Engine Tutorial).	Script executes and produces the expected result with no error.
6	UltraEdit is the focused window	Click into the editor pane before testing.	Title bar shows UltraEdit as the active application.

If any check fails, fix the script registration before proceeding to chord assignment. A chord cannot trigger a script that does not run manually.

3. Entering the Multi-Key Chord in the Scripting Dialog

Once `run_selection` is in the Scripts list and runs manually:

Step-by-step

1. Open **Scripting** → **Scripts** (Advanced tab → **All scripts**).
2. Locate `run_selection` in the script list.
3. Click in the **Hotkey** field for that script row. The field becomes editable with a visible cursor ([UltraEdit Wiki: Scripts](#); [UltraEdit Forums](#)).
4. Press the first key combination (e.g. `Ctrl+K`). Do not type the letters C-t-r-l-K — press the actual keys. UltraEdit captures and displays the combination.
5. Click in the **Chord** field for the same script row.
6. Press the second key combination (e.g. `Ctrl+R`). Again, press the physical keys.
7. Click **OK** to save.

Important rules

- **Hotkey first, then Chord.** The official tutorial states: "You'll need to create a hotkey first before you can set up a chord" ([UltraEdit Scripting Engine Tutorial](#)). If the Chord field does not accept input, verify the Hotkey field is populated.
- **Press, don't type.** The fields capture physical key presses. Typing the text `Ctrl+K` will not work.
- **Click away to confirm.** After entering a key in either field, click elsewhere in the dialog to ensure the value is committed before clicking OK ([UltraEdit Forums](#)).
- **Avoid arrow-key and special-key hotkeys if possible.** A known UltraEdit bug causes certain key combinations (e.g. `Alt+Shift+Left`) to display as `Alt+Shift+EXECUTE` and stop responding after the dialog closes, even though they appeared correct initially ([UltraEdit Forums](#)). Stick to `Ctrl/Shift` + letter combinations for script hotkeys and chords.

How the chord works at runtime

When you press the Hotkey combination, UltraEdit starts a timer (the Multi-key delay). If you press the Chord combination within that window, the script runs. If you do not press the Chord in time, the action bound to the Hotkey alone (if any) executes instead ([UltraEdit Wiki: Key mapping](#)).

4. Debugging: When the Chord Does Not Trigger

Work through these steps in order. Each step isolates a different failure point.

Step 1 — Does the script run manually?

Return to **Scripting** → **Scripts**, select `run_selection`, and run it directly from the menu. If it fails or errors, the problem is in the script, not the chord.

Step 2 — Is the chord visibly saved?

Reopen **Scripting** → **Scripts** and inspect the Hotkey and Chord fields for `run_selection`.

- If either field is blank or shows garbled text (e.g. `Alt+Shift+EXECUTE`), the assignment was not captured correctly — re-enter it per Section 3 ([UltraEdit Forums](#)).
- If the values look correct, proceed to Step 3.

Step 3 — Check the Multi-key Delay

The Multi-key delay controls how long UltraEdit waits for the second key of a chord after the first is pressed. If it is too short, the editor gives up before you can complete the sequence.

1. Go to **Settings** → **Key Mapping** (in classic menus: **Advanced** → **Configuration** → **Key Mapping**) ([UltraEdit Wiki: Key mapping](#)).
2. Locate the **Multi-key delay** field. It is expressed in **milliseconds** ([UltraEdit Wiki: Key mapping](#)).

3. If the value is very low (e.g. under 300 ms), increase it. A value of 500–1000 ms gives comfortable time to press the second key combination.
4. Click **OK** to save.
5. Test the chord again.

If the first key combination triggers an action on its own (without you pressing the second), the delay is too short or the first key combination is also bound to another command as a single-key shortcut.

Step 4 — Check for conflicting hotkey assignments

A chord may fail to fire if the first key combination is already assigned to another command as a standalone shortcut — when the Multi-key Delay expires, that standalone command runs instead of waiting for the second key. An exact duplicate of the full chord on another command, or a conflicting first-stroke prefix, can also interfere. Even a standalone assignment on just the second stroke is worth checking, as it can produce unexpected behavior. Scan the generated shortcut list for any of these overlaps.

1. In **Settings** → **Key Mapping**, scan the **Commands** list for anything using your chosen keys ([UltraEdit Wiki: Key mapping](#)).
2. Use the **Show Key Mapping in Editor** button (also called "Generate shortcut list") to produce a plain-text file of all currently configured shortcuts. The file is named `[EditorName].shortcuts.txt` and opens automatically ([UltraEdit Wiki: Key mapping](#)).
3. Search the generated file for your Hotkey and Chord combinations.
4. If either combination appears under another command, remove the conflicting assignment:
 - Select the command in the **Commands** list.
 - Select the existing key in the **Existing keys** combo box.
 - Click **Remove**.
 - Click **Assign** to bind any replacement if needed.
 - Click **OK** to save ([UltraEdit Wiki: Key mapping](#)).
5. Click **OK** and retest.

Step 5 — Test with a simple, unused chord

To distinguish a script failure from a keybinding failure, temporarily assign a chord using simple, unlikely-to-conflict keys (e.g. **Ctrl+Q** as the Hotkey, **Ctrl+W** as the Chord).

- If the simple chord works, the original chord had a conflict — return to Step 4.
- If the simple chord also fails, the problem is with the script registration or the Scripting dialog itself — return to Step 1.

Step 6 — Restart UltraEdit

After confirming the mapping looks correct in both the Scripts dialog and Key Mapping, close and reopen UltraEdit. The keyboard-shortcut file (stored as Uedit32.uek, Uedit64.uek, or UESstudio.uek in the same directory as your INI file, typically %APPDATA%\IDMComp\UltraEdit) is loaded at startup ([UltraEdit Wiki: Key mapping](#)). A restart ensures the saved mappings are freshly loaded.

Step 7 — Check for version-specific script-hotkey capture issues

If your Hotkey or Chord involves arrow keys or other special keys, you may be hitting a reported, version-specific UltraEdit issue where the Scripts dialog corrupts the assignment on save. Symptoms:

- The key appears correct in the dialog but shows as Alt+Shift+EXECUTE after reopening ([UltraEdit Forums](#)).
- The same key combination works fine when assigned to an editor command via **Settings → Key Mapping** ([UltraEdit Forums](#)).

Workaround: Switch to a Ctrl+letter or Ctrl+Shift+letter combination for the script Hotkey and Chord, avoiding arrow keys, Tab, and Enter, which are more prone to capture anomalies in the Scripts dialog.

5. Quick-Reference Summary

Task	Correct location	Wrong location
Add <code>run_selection.js</code> as a script	Scripting → Scripts dialog	Tool Configuration
Assign Hotkey to a script	Scripting → Scripts → Hotkey field	Tool Configuration (for external commands only)
Assign Chord to a script	Scripting → Scripts → Chord field (after Hotkey is set)	Tool Configuration
Adjust Multi-key delay	Settings → Key Mapping	Scripting → Scripts
Check for conflicting shortcuts	Settings → Key Mapping → Show Key Mapping in Editor	Scripting → Scripts
Add an external command-line tool	Advanced → Tool Configuration	Scripting → Scripts
Map global editor/macro/tool commands	Settings → Key Mapping	Scripting → Scripts

Sources

- [UltraEdit Wiki: Scripts](#)
- [UltraEdit Wiki: Settings — Key mapping](#)
- [UltraEdit Wiki: Configure tools](#)
- [UltraEdit Scripting Engine Tutorial](#)
- [UltraEdit Forums: Script hotkeys — Alt+Shift+Left and Alt+Shift+Right](#)
- [UltraEdit Forums: Can a script be run with a keyboard shortcut?](#)
- [UltraEdit Wiki: Script list](#)